

Inteligencia Artificial Generativa Aplicada al Análisis de
Datos

Tema 5. Ingeniería de características con IA generativa

Índice

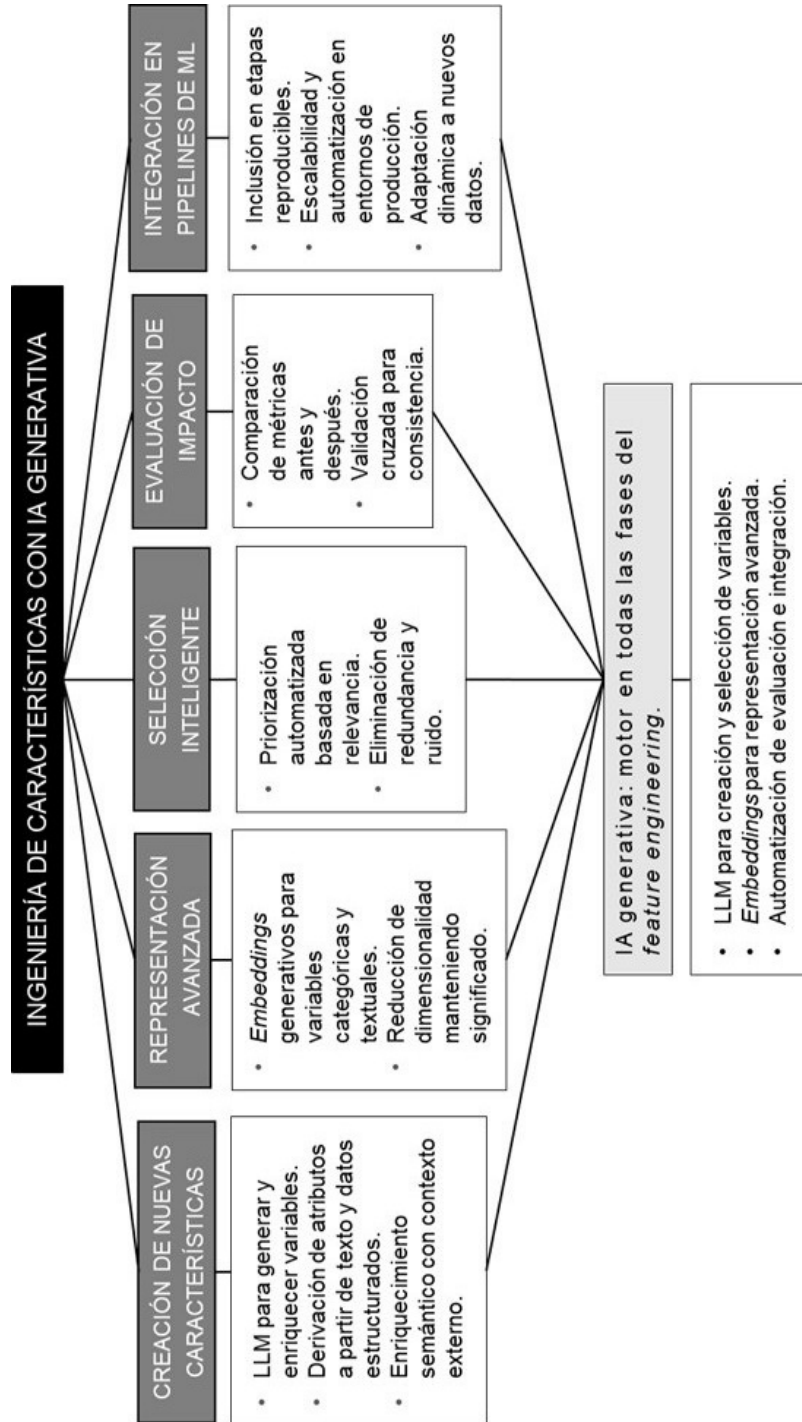
Esquema

Ideas Clave

- 5.1. Introducción y objetivos
- 5.2. Fundamentos de ingeniería de características
- 5.3. Generación de nuevas variables con LLM
- 5.4. Embeddings generativos para variables categóricas
- 5.5. Selección de características e importancia de variables con IA generativa
- 5.6. Evaluación del Impacto en modelos predictivos
- 5.7. Feature engineering en pipelines de machine learning
- 5.8. Comparación con técnicas tradicionales
- 5.9. Exploración práctica
- 5.10. Referencias bibliográficas

A fondo

- Ingeniería de características automatizada con LLMs
- Un enfoque de diálogo con LLMs para la creación de características



5.1. Introducción y objetivos

En los temas anteriores nos hemos centrado en enriquecer y completar los datos, ya sea generando nuevos registros o rellenando información ausente. Ahora avanzamos hacia una de las fases más determinantes en el éxito de cualquier modelo de aprendizaje automático: la ingeniería de características («*feature engineering*»). Esta etapa consiste en transformar los datos crudos disponibles en variables útiles, relevantes y expresivas que permitan a un modelo capturar patrones predictivos con precisión.

Domingos (2012) sostiene que la clave del éxito en el aprendizaje automático no reside únicamente en el algoritmo, sino en cómo se representan los datos. Esta afirmación cobra aún más sentido en la práctica profesional; dos modelos idénticos pueden ofrecer rendimientos drásticamente distintos si trabajan con diferentes conjuntos de características. En esencia, un buen modelo con malas variables es inútil; un modelo sencillo con buenas variables puede ser extraordinario.

La ingeniería de características ha sido tradicionalmente una labor artesanal: los expertos analizan los datos, exploran combinaciones lógicas, aplican transformaciones matemáticas o de dominio y, con suerte, generan nuevas variables con mayor poder predictivo. Este enfoque, aunque eficaz, es costoso, lento y difícil de escalar.

La IA generativa, especialmente a través de modelos de lenguaje grandes (LLMs), está revolucionando esta práctica. Ahora es posible delegar parte del proceso a sistemas automatizados capaces de proponer transformaciones lógicas o matemáticas relevantes para el problema en cuestión, crear nuevas variables semánticas a partir de texto libre utilizando comprensión contextual avanzada, generar *embeddings* numéricos para variables categóricas o texto, codificando relaciones latentes no evidentes y evaluar y seleccionar automáticamente las variables más relevantes en función del rendimiento del modelo.

Como ya se ha destacado en otros temas, lejos de reemplazar al experto, estos modelos actúan como copilotos. Este enfoque es coherente con la visión del analista aumentado, ya presentado, o *augmented analyst*, donde la creatividad y la intuición humanas se combinan con la capacidad generativa y contextual de la IA.

Al finalizar este tema, el estudiante será capaz de:

- ▶ Objetivo número 1. Comprender las técnicas generativas para proponer, transformar y enriquecer variables predictivas.
- ▶ Objetivo número 2. Integrar la ingeniería de características dentro de pipelines de Machine Learning reproducibles y escalables.
- ▶ Objetivo número 3. Conocer los diferentes criterios de evaluación para medir y cuantificar el impacto (uplift) de las características generadas, justificando su valor de negocio.

5.2. Fundamentos de ingeniería de características

La ingeniería de características es el proceso mediante el cual los datos crudos se transforman en representaciones que hacen que los algoritmos de Machine Learning sean más eficaces y precisos. Esta fase, ampliamente reconocida como una de las más determinantes en el rendimiento de un modelo, busca maximizar la señal informativa y reducir el ruido. Como afirma Domingos (2012), ya mencionado, una buena representación de los datos es tan importante como el algoritmo e incluso puede determinar el éxito del aprendizaje automático.

En términos prácticos, este proceso implica modificar el espacio de representación del problema: es decir, transformar la manera en que se codifican los datos para que los algoritmos puedan aprender relaciones más significativas. Mientras que los modelos aprenden patrones a partir de las variables que se les dan, no pueden inferir lo que no está explícitamente representado. Por ello, la ingeniería de características se sitúa en el corazón de cualquier pipeline de modelado robusto (Kuhn y Johnson, 2019).

Dentro de las principales operaciones de la ingeniería de características encontramos:

- ▶ Construcción de nuevas variables: a partir de datos existentes, como calcular la edad a partir de la fecha de nacimiento o la ratio entre dos cantidades económicas (por ejemplo, deuda/ingresos).
- ▶ Transformaciones matemáticas o estadísticas: escalado (min-max, z-score); transformaciones logarítmicas; o generación de variables cíclicas para variables temporales (por ejemplo, codificar el mes del año como un par de coordenadas).

- ▶ Codificación de variables categóricas: aplicación de técnicas como one-hot encoding, ordinal encoding o embeddings para traducir categorías en representaciones numéricas comprensibles por los modelos.
- ▶ Reducción o combinación de variables: mediante técnicas como el Análisis de Componentes Principales (PCA), la selección de variables basada en su importancia (por ejemplo, mediante árboles de decisión), o la generación de nuevas variables a través de combinaciones no lineales supervisadas (como redes neuronales o autoencoders).

Estas técnicas se basan en un conocimiento profundo tanto del problema como del técnico de modelado, así como de los datos disponibles, lo que explica por qué, tradicionalmente, esta labor ha requerido la intervención de humanos. Sin embargo, con la llegada de la IA generativa, esta frontera está comenzando a desplazarse, permitiendo que algunos de estos procesos se automaticen de manera guiada o autónoma, como se explora en los siguientes apartados.

5.3. Generación de nuevas variables con LLM

La generación de nuevas características con modelos de lenguaje de gran escala (LLM) representa un avance fundamental en la ingeniería de características al incorporar conocimiento externo y razonamiento simbólico en el diseño de variables predictivas. A diferencia de las técnicas tradicionales, que operan exclusivamente sobre el contenido del dataset mediante transformaciones estadísticas o algebraicas, los LLMs pueden integrar lógica de negocio, conocimiento del dominio y experiencia previa codificada en lenguaje natural.

Este enfoque pertenece a lo que se conoce como Feature Engineering basado en conocimiento (Knowledge-Based Feature Construction), una técnica que históricamente ha dependido del criterio humano, pero que ahora puede ser parcialmente automatizada. Según Boonstra (2024), un LLM puede actuar como motor de síntesis —capaz de recibir una descripción en lenguaje natural sobre una necesidad de análisis y traducirla en código funcional reproducible—.

La ingeniería de características basada en LLMs introduce una nueva forma de pensar y operar sobre los datos: una que combina la intuición humana, el conocimiento del dominio y la capacidad computacional del lenguaje natural. Tradicionalmente, como ya se ha mencionado, este proceso ha sido manual, iterativo y profundamente dependiente del conocimiento del científico de datos. Sin embargo, los modelos de lenguaje como GPT-4 o Claude pueden ahora participar activamente en la síntesis, transformación y documentación de nuevas variables, abriendo paso a una ingeniería de características aumentada.

Los enfoques tradicionales se basan en estadísticas, conocimiento del dominio y transformación algebraica de las variables. Un analista podía derivar variables útiles, pero con una limitación clara: el proceso era lento, sujeto a sesgos y poco escalable. En cambio, los LLMs permiten automatizar y acelerar este proceso, con una capacidad sorprendente de razonamiento simbólico y generación de código.

El proceso se basa en tres pasos fundamentales:

- ▶ Descripción en lenguaje natural de la lógica que define una nueva variable (por ejemplo: «Si un cliente lleva más de 3 años y su gasto mensual es mayor que 100 €, se considera premium»).
- ▶ Procesamiento por el LLM, que interpreta la lógica y propone una fórmula o script en Python, SQL u otro lenguaje de análisis de datos.
- ▶ Validación y ajuste por parte del analista, que revisa si la lógica es correcta, útil y coherente con el objetivo del modelo.

Este enfoque permite generar características que, de otro modo, podrían no surgir de un simple análisis estadístico. Los LLM destacan especialmente en los siguientes tipos de variables:

A) Variables de interacción semántica

Los LLM pueden generar nuevas características que combinan múltiples columnas de forma significativa sin requerir al analista que pruebe exhaustivamente todas las combinaciones posibles.

Ejemplo:

Prompt:

Crea una variable llamada `precio_por_metro_cuadrado` dividiendo el precio entre los metros cuadrados.

O aún más compleja:

A partir de ingresos y educación, inferir una categoría socioeconómica. El LLM puede interpretar la lógica de segmentación y codificarla directamente.

B) Variables condicionales o basadas en lógica

Los LLM pueden aplicar reglas complejas con múltiples condiciones anidadas para etiquetar o clasificar observaciones.

Ejemplo:

«Genera una columna llamada `es_cliente_premium` que sea 1 si antigüedad > 3 años y gasto mensual > 100€, y 0 en caso contrario».

El LLM traduce instrucciones en lenguaje natural a código ejecutable.

C) Transformaciones guiadas por contexto matemático o semántico

Los LLM también pueden sugerir transformaciones matemáticas o estadísticas sobre variables si se les proporciona contexto. También permiten sugerencias automáticas con base en la descripción del problema.

Ejemplo:

«La variable ingresos tiene una distribución muy sesgada a la derecha».

Prompt:

Sugiere una transformación para reducir la asimetría positiva en una variable de ingresos altamente sesgada.

Respuesta esperada:

«Una transformación logarítmica como $\log(\text{ingresos} + 1)$ puede reducir la asimetría y estabilizar la varianza».

Esta capacidad de los LLM para traducir lenguaje natural en operaciones complejas y reproducibles acelera el proceso de ingeniería de características, aumenta su cobertura (más variables, más pruebas) y democratiza su uso. Además, estos modelos pueden incorporar información externa (como definiciones legales, conocimiento sectorial o comportamientos históricos) que no está contenida en los datos y permite generar variables enriquecidas de alto valor predictivo.

La tabla 1 muestra las principales diferencias con las metodologías tradicionales de feature engineering.

Dimensión	Técnicas tradicionales	Ingeniería con LLM
Entrada	Código manual, intuición estadística.	Lenguaje natural (instrucciones, descripciones).
Tiempo de desarrollo	Alto.	Bajo (respuestas inmediatas con ajustes menores).
Creatividad y cobertura	Limitada por experiencia individual.	Ampliada por conocimiento global y ejemplos previos.
Trazabilidad	Manual (anotaciones, <i>docstrings</i>).	Documentación automática integrada.
Adaptabilidad a nuevos dominios	Requiere capacitación específica.	Generaliza fácilmente a nuevos sectores y lógicas.
Validación técnica	Estadística y prueba manual.	Requiere revisión humana experta.
Riesgos	Omisión de variables clave.	Alucinaciones, reglas incorrectas si es mal instruido.

Tabla 1. Comparativa técnica: LLM vs técnicas tradicionales. Fuente: elaboración propia.

A modo de ejemplo, mostraremos cómo un LLM puede ayudar a transformar descripciones en lenguaje natural en variables útiles para un modelo predictivo.

En primer lugar, en un escenario de predicción de churn en telecomunicaciones, el equipo sospecha que los clientes con bajo consumo de datos y alta interacción con soporte tienden a abandonar antes. La idea es capturar esta información en una nueva variable. En lugar de codificar la variable, pedimos al LLM que devuelva el código necesario para crearla.

```
import os, textwrap
import numpy as np, pandas as pd
from openai import OpenAI
# Configura tu clave
# os.environ["OPENAI_API_KEY"] = "PON_AQUI_TU_CLAVE"
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

# Dataset de ejemplo
np.random.seed(7)
df = pd.DataFrame({
    "antiguedad_meses": np.random.randint(1, 60, 30),
    "consumo_gb": np.random.gamma(shape=4, scale=4, size=30).round(2),
    "llamadas_soporte": np.random.poisson(lam=2.0, size=30),
    "plan_contratado": np.random.choice(["Basico", "Estandar", "Premium"], size=30, p=[0.5, 0.3, 0.2])
})

prompt = textwrap.dedent("""
Crea una nueva columna en el DataFrame X llamada 'friccion_reciente'
a partir de las columnas 'consumo_gb' y 'llamadas_soporte'.
La variable debe capturar la idea de fricción reciente: alto número de llamadas con bajo consumo.
No uses imports ni accesos a sistema, solo transforma X en el lugar.
""")

rsp = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.0
)

code = rsp.choices[0].message.content.strip()
X = df.copy()
local_vars = {"X": X, "np": np, "pd": pd}
exec(code, {}, local_vars)
X = local_vars["X"]

X[["consumo_gb", "llamadas_soporte", "friccion_reciente"]].head(8)
```

El resultado es una columna adicional que marca con valores binarios o categóricos a los clientes con baja utilización y alta interacción con soporte. Una muestra de los registros iniciales puede verse en la tabla 2:

consumo_gb	llamadas_soporte	friccion_reciente
11.21	2	0
21.94	0	0
12.72	3	1
7.43	5	1
25.18	1	0

Tabla 2. Muestra de datos de ejemplo. Fuente: elaboración propia.

Se observa que los clientes con bajo consumo y varias llamadas son clasificados como de fricción alta, lo que concuerda con la hipótesis planteada.

5.4. Embeddings generativos para variables categóricas

Una de las tareas más complejas en el preprocesamiento de datos tabulares es el tratamiento eficaz de variables categóricas. El enfoque clásico más extendido ha sido el One-Hot Encoding, que convierte cada categoría en una columna binaria.

Si bien este método es sencillo y ampliamente compatible con algoritmos tradicionales, tiene limitaciones:

Alta dimensionalidad: cuando el número de categorías es grande (como códigos postales, productos, países), el espacio de representación se vuelve inmanejable, especialmente para modelos sensibles a la dimensión como árboles o redes neuronales.

Ausencia de semántica: el One-Hot trata todas las categorías como equidistantes y sin relación. No distingue que, por ejemplo, los productos «camisa» y «pantalón» son más similares entre sí que «camisa» y «móvil».

Este problema ha sido abordado por técnicas más sofisticadas como el Target Encoding o la codificación ordinal supervisada, pero su alcance es limitado cuando hay pocas muestras por categoría o cuando se desea capturar relaciones semánticas ricas.

En resumen, mientras que el One-Hot Encoding, Label Encoding o incluso los embeddings preentrenados para texto o categorías han sido técnicas estándar durante años, los LLMs ahora permiten:

- ▶ Explicar automáticamente cuál es la codificación más adecuada según el tipo de variable.
- ▶ Sugerir embeddings semánticos para variables categóricas complejas.

- ▶ Evaluar redundancia o colinealidad entre variables derivadas.
- ▶ Recomendar eliminar transformaciones irrelevantes o poco robustas.

Por ejemplo, en lugar de aplicar One-Hot a una columna «categoría producto» con 150 valores únicos (lo que produce alta dimensionalidad), el LLM puede sugerir usar embeddings aprendidos o agrupar categorías de forma semántica o jerárquica.

Pero ¿qué son los embeddings? Los embeddings son representaciones vectoriales densas y de baja dimensión que ubican elementos discretos (como palabras, productos o categorías) en un espacio continuo. Su objetivo es que elementos semánticamente similares tengan vectores cercanos entre sí.

En lugar de una codificación binaria dispersa (sparse), los embeddings permiten trabajar con una representación distribuida que captura coocurrencias, similitud semántica y relaciones contextuales. Esta técnica es esencial en NLP y visión computacional, y su adopción en datos tabulares ha crecido notablemente gracias a su versatilidad.

Una de las formas más eficaces de generar embeddings para variables categóricas es mediante un autoencoder, una red neuronal diseñada para reconstruir sus entradas después de comprimirlas en un espacio latente de baja dimensión. El proceso comienza representando la categoría mediante técnicas estándar como One-Hot Encoding o codificación por índices. A continuación, se entrena la red con un cuello de botella (bottleneck) en su arquitectura: este embudo obliga al modelo a sintetizar la información esencial necesaria para predecir el resto de las variables del dataset. El resultado es que el vector intermedio en esa capa latente se convierte en el embedding numérico aprendido para cada categoría.

Este enfoque tiene varias ventajas destacables. A diferencia de los embeddings preentrenados en corpus genéricos, estos vectores se adaptan al dominio específico de los datos. Además, se benefician de una semántica emergente, donde categorías

empleadas en contextos similares tienden a ocupar posiciones cercanas en el espacio latente. Cuando se entrena el autoencoder con una señal supervisada (por ejemplo, incluyendo la variable objetivo en la reconstrucción), los embeddings resultantes pueden optimizarse explícitamente para mejorar la predicción.

Como señala Chollet (2021), este tipo de arquitectura no solo reduce la dimensionalidad, sino que también obliga a la red a descubrir patrones internos relevantes que quedarían ocultos en codificaciones tradicionales como el one-hot. En resumen, los autoencoders permiten representar las categorías no como etiquetas planas, sino como puntos en un espacio que reflejan sus relaciones contextuales aprendidas desde los propios datos.

Ejemplo:

Supón un conjunto de datos de clientes con una columna ocupación (categoría) y otras variables como ingresos, edad y compra. Podemos usar un autoencoder para representar ocupación como vector de 3 dimensiones:

```
from sklearn.preprocessing import OneHotEncoder
from keras.models import Model
from keras.layers import Input, Dense
import pandas as pd

# Datos ficticios
df = pd.DataFrame({'ocupacion': ['médico', 'ingeniero', 'artista', 'médico', 'artista'],
                  'ingresos': [5000, 4000, 3000, 5500, 3100],
                  'edad': [45, 38, 30, 47, 32]})

# One-Hot Encoding
ohe = OneHotEncoder(sparse_output=False)
X_cat = ohe.fit_transform(df[['ocupacion']])

# Autoencoder para generar embeddings
input_dim = X_cat.shape[1]
encoding_dim = 3 # número de dimensiones del embedding

input_layer = Input(shape=(input_dim,))
encoded = Dense(encoding_dim, activation='relu')(input_layer)
decoded = Dense(input_dim, activation='sigmoid')(encoded)
```



```
autoencoder = Model(input_layer, decoded)
autoencoder.compile(optimizer='adam', loss='mse')
autoencoder.fit(X_cat, X_cat, epochs=100, verbose=0)

# Extraer embeddings
encoder = Model(input_layer, encoded)
ocupacion_embeddings = encoder.predict(X_cat)

print(pd.DataFrame(ocupacion_embeddings))
```

Este vector numérico de 3 dimensiones reemplaza la representación dispersa inicial y puede alimentar cualquier modelo de clasificación o regresión. Si entrenamos el autoencoder junto con otras variables del dataset, estos vectores reflejarán similitudes funcionales: por ejemplo «médico» y «enfermero» podrían tener embeddings cercanos.

Los modelos generativos de lenguaje como GPT-4 también pueden sugerir cómo agrupar categorías, transformar texto categórico complejo o generar embeddings textuales directamente. Si una categoría es una descripción larga («Empleado de banca con experiencia en seguros»), el LLM puede resumirla o mapearla automáticamente a una representación vectorial utilizando técnicas como sentence-transformers o text-embedding-ada-002 (un modelo de OpenAI diseñado específicamente para generar embeddings de texto, es decir, representaciones numéricas densas de frases, palabras, documentos, etc., que capturan su significado semántico).

Ejemplo:

```
from openai import OpenAI
client = OpenAI(api_key="TU_API_KEY")

response = client.embeddings.create(
    model="text-embedding-ada-002",
    input=["médico", "ingeniero", "artista"]
)
```

Este tipo de *embeddings* captura conocimiento externo sobre las profesiones (están preentrenados en grandes corpus de texto multilingües) y resulta útil cuando las categorías no están bien representadas en nuestros datos.

Los embeddings generativos representan un avance crucial frente al One-Hot Encoding: comprimen, generalizan y capturan relaciones semánticas profundas. Ya sea aprendidos desde los datos (con autoencoders) o generados desde texto por LLM, estos vectores ofrecen una forma más rica y flexible de representar variables categóricas.

Este enfoque mejora sustancialmente el rendimiento de modelos de machine learning en escenarios con alta cardinalidad o información textual no estructurada. Los embeddings aprendidos directamente desde los datos o con soporte de modelos generativos no solo reducen la dimensionalidad, sino que permiten una codificación semántica adaptada al contexto de uso, especialmente en tareas donde las relaciones entre categorías no son triviales. En consecuencia, los embeddings mejoran la generalización y el rendimiento en conjuntos de datos con muchas categorías.

Como hemos visto, a modo de ejemplo, cuando se trabaja con datos de clientes, no todas las variables provienen de números o categorías predefinidas. Muchas veces existen campos de texto libre, como notas de soporte o comentarios, que contienen información relevante para la predicción de abandono, satisfacción o intención de compra. Los LLM permiten transformar estos textos en nuevas variables, primero de manera categórica interpretable (por ejemplo, un sentimiento positivo/negativo) y luego como representaciones densas en forma de embeddings que capturan similitudes semánticas. De esta manera, partimos de frases cortas de soporte al cliente, las clasificamos en categorías de sentimiento usando un LLM y, posteriormente, generamos embeddings vectoriales para cada frase. Se aprecia cómo la información textual pasa de ser texto libre a variables listas para modelos predictivos.

```
import os, json
import pandas as pd
from openai import OpenAI

# Configura tu clave
# os.environ["OPENAI_API_KEY"] = "PON_AQUI_TU_CLAVE"
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

# Dataset de frases de soporte
tickets = pd.DataFrame({
    "id": [f"T{i:03d}" for i in range(1,7)],
    "nota_soporte": [
        "Muy satisfecho con la velocidad. Gracias.",
        "La factura llegó incorrecta otra vez.",
        "No puedo activar el roaming y nadie responde.",
        "Me resolvieron la portabilidad sin problemas.",
        "La app se cierra y llevo dos días así.",
        "Excelente atención hoy."
    ]
})

# Prompt de clasificación de sentimiento
SYSTEM = "Eres un analista conciso. Responde SOLO con JSON válido."
USER_TMPL = ""
Clasifica el sentimiento del siguiente texto de ticket de cliente como NEGATIVO, NEUTRAL o POSITIVO.
Devuelve un JSON con las claves exactas: {"sentimiento": "...", "justificacion": "..."}

```

Texto: "{texto}"

"""

```
def clasificar_sentimiento(texto: str) -> dict:
    rsp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "SYSTEM"},
            {"role": "user", "content": USER_TMPL.format(texto=texto)}
        ],
        temperature=0.0
    )
    content = rsp.choices[0].message.content
    try:
        return json.loads(content)
    except:
        return {"sentimiento": "NEUTRAL", "justificacion": "Formato inesperado"}

# Aplicar la clasificación
out = tickets["nota_soporte"].apply(clasificar_sentimiento).apply(pd.Series)
tickets = pd.concat([tickets, out], axis=1)
tickets[["id", "nota_soporte", "sentimiento"]]
```

Como resultado, cada frase queda asociada a una etiqueta de sentimiento. A partir de ello, se cuenta con una variable categórica interpretable. Sin embargo, es posible enriquecer aún más la representación generando vectores de incrustación, es decir, vectores densos que resumen el contenido semántico de cada frase.

```
# Generar embeddings
embeddings = []
for txt in tickets["nota_soporte"]:
    emb = client.embeddings.create(
        model="text-embedding-3-small",
        input=txt
    )
    embeddings.append(emb.data[0].embedding)

tickets["embedding"] = embeddings

# Mostrar primeras dimensiones para ilustrar
tickets["embedding_preview"] = tickets["embedding"].apply(lambda v: [round(x,3) for x in v[:5]])
tickets[["id", "sentimiento", "embedding_preview"]]
```

La tabla 3 combina la etiqueta de sentimiento con una vista parcial de los embeddings.

id	sentimiento	embedding_preview
T001	POSITIVO	[0.012, -0.034, 0.087, 0.041, -0.022]
T002	NEGATIVO	[-0.056, 0.103, -0.024, 0.065, 0.011]
T003	NEGATIVO	[-0.044, 0.095, -0.012, 0.058, 0.014]
T004	POSITIVO	[0.018, -0.029, 0.092, 0.047, -0.019]
T005	NEGATIVO	[-0.061, 0.099, -0.020, 0.071, 0.009]
T006	POSITIVO	[0.016, -0.031, 0.090, 0.043, -0.021]

Tabla 3. Muestra de datos con embedding. Fuente: elaboración propia.

Aunque solo se muestran las primeras cinco dimensiones, cada vector completo posee cientos de valores que ubican la frase en un espacio semántico multidimensional. Las frases con contenido similar tienden a generar embeddings cercanos entre sí.

El ejemplo muestra cómo pasar de un campo textual desestructurado a dos niveles de representación: primero, una variable categórica fácilmente interpretable (sentimiento), y después, un vector denso que busca preservar la semántica de cada frase. Este doble enfoque resulta valioso. Por un lado, la variable categórica facilita la comunicación con áreas de negocio: es claro distinguir entre clientes satisfechos y frustrados. Por otro, los embeddings ofrecen una representación más rica para los modelos predictivos, capaces de detectar matices y similitudes entre frases que trascienden etiquetas discretas.

En síntesis, los embeddings constituyen una herramienta esencial para integrar datos textuales en pipelines de machine learning, mejorando la capacidad de los modelos de capturar patrones de lenguaje y, en consecuencia, de predecir con mayor robustez.

5.5. Selección de características e importancia de variables con IA generativa

En la ingeniería de características, una vez ampliado el espacio de variables, ya sea por transformaciones, combinaciones o generación asistida por IA, el siguiente paso crítico es seleccionar las características más relevantes para el modelo. Esta etapa no solo optimiza el rendimiento predictivo, sino que mejora la interpretabilidad, reduce el riesgo de sobreajuste (overfitting), disminuye la carga computacional y facilita la trazabilidad de los resultados.

La feature importance se refiere a la cuantificación de la influencia que tiene cada variable de entrada sobre la predicción de un modelo. Esta importancia no es intrínseca a los datos, sino relativa al algoritmo y al objetivo que se persigue. Su estimación adecuada permite construir modelos más precisos, eficientes y comprensibles.

Dos enfoques tradicionales han dominado esta tarea:

- ▶ Importancia basada en árboles: algoritmos como Random Forest o XGBoost calculan la ganancia media de información (o reducción de impureza) que produce cada variable al dividir nodos del árbol. Es rápido y accesible, pero puede presentar sesgo hacia variables numéricas de alta cardinalidad.
- ▶ Importancia por permutación: evalúa el impacto en el rendimiento del modelo al desordenar una sola variable, midiendo cuánto se degrada la predicción. Este método, agnóstico al modelo, es más robusto, pero computacionalmente costoso.

A estas técnicas se suman otras como la regresión Lasso (para modelos lineales), la eliminación recursiva de características (RFE), o el uso de valores de Shapley, implementados mediante SHAP (Shapley additive explanations), que proporcionan interpretabilidad local y global basada en teoría de juegos (Lundberg y Lee, 2017).

Se ha demostrado que la asignación justa y consistente de importancia basada en valores de Shapley permite no solo interpretar los modelos, sino también identificar variables redundantes o poco informativas con un fuerte respaldo teórico y empírico.

Sin embargo, la incorporación de modelos de lenguaje como copilotos de análisis transforma radicalmente esta fase. La IA generativa no sustituye los métodos anteriores, sino que los potencia a través de tres grandes capacidades:

- ▶ Como generador de código: LLMs como GPT-4 pueden generar código completo y adaptado al contexto del dataset para aplicar técnicas avanzadas de selección, combinando métodos estadísticos y heurísticos en pipelines reproducibles.
- ▶ Como experto de dominio: a diferencia de los métodos puramente estadísticos, un LLM puede incorporar conocimiento del dominio (explícito o inferido) y proponer qué variables deberían incluirse, fusionarse o descartarse basándose en objetivos estratégicos, interpretabilidad esperada o restricciones del negocio.
- ▶ Como intérprete de resultados: la IA generativa puede interpretar los resultados de análisis de importancia y convertirlos en explicaciones comprensibles para perfiles no técnicos, facilitando la colaboración entre ciencia de datos y áreas funcionales.

Como destaca Boonstra (2024), el papel emergente del LLM no es el de un filtro estadístico, sino el de un «curador semántico» del espacio de variables, guiando la construcción del modelo hacia estructuras más alineadas con el conocimiento experto y las metas organizacionales.

De esta manera, en la práctica, no todas las variables generadas aportan valor. Como se ha visto, es fundamental aplicar métodos de selección de características para identificar cuáles contribuyen de forma relevante al desempeño del modelo. Como ejemplo, trabajamos con un dataset simulado de clientes de telecomunicaciones para predecir la variable churn (abandono). Se comparará la importancia relativa de las variables originales (`antigüedad_meses`, `consumo_gb`,

llamadas_soporte) con dos variables creadas en apartados anteriores: fricción_reciente (regla de negocio generada por LLM) y sentimiento (extraído de notas de soporte y convertido en categorías).

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split

# Dataset simulado
np.random.seed(42)
n = 300
df = pd.DataFrame({
    "antiguedad_meses": np.random.randint(1, 60, n),
    "consumo_gb": np.random.gamma(shape=4, scale=4, size=n).round(2),
    "llamadas_soporte": np.random.poisson(lam=2.0, size=n),
    "plan_contratado": np.random.choice(["Basico", "Estandar", "Premium"], size=n, p=[0.5, 0.3, 0.2]),
})

# Variable de churn (sintética)
df["churn"] = ((df["llamadas_soporte"] > 3) & (df["consumo_gb"] < 10)).astype(int)

# Variables generadas
df["friccion_reciente"] = ((df["consumo_gb"] < 12) & (df["llamadas_soporte"] >= 3)).astype(int)
df["sentimiento"] = np.random.choice(["POSITIVO", "NEGATIVO"], size=n, p=[0.6, 0.4])

# One-hot encoding
X = pd.get_dummies(df.drop(columns=["churn"]), drop_first=True)
y = df["churn"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Modelo
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)

# Importancia de variables
imp = pd.DataFrame({
    "variable": X.columns,
    "importancia": rf.feature_importances_
}).sort_values("importancia", ascending=False)

imp.head(10)
```

Una muestra de los resultados típicos de importancia podría verse en la tabla 4.

variable	importancia
friccion_reciente	0.31
llamadas_soporte	0.24
consumo_gb	0.19
antiguedad_meses	0.11
sentimiento_NEGATIVO	0.08
plan_contratado_Premium	0.04
plan_contratado_Estandar	0.03

Tabla 4. Importancia relativa ejemplo. Fuente: elaboración propia.

Se observa que la nueva variable `friccion_reciente` aparece como la más influyente, incluso por encima de `llamadas_soporte` y `consumo_gb`. La categoría `sentimiento_NEGATIVO` también aporta valor, aunque en menor medida.

Este ejemplo muestra que las variables generadas mediante LLM no solo, sino que pueden convertirse en predictoras clave al integrarse en modelos. El análisis de importancia muestra que `friccion_reciente`, que combina información de consumo y llamadas, aporta una señal fuerte asociada al abandono, mientras que el sentimiento refuerza la explicación al capturar actitudes de los clientes. Esto justifica su inclusión en un pipeline de modelado.

5.6. Evaluación del Impacto en modelos predictivos

La ingeniería de características no es solo una tarea creativa; su utilidad debe medirse rigurosamente. La validación cuantitativa del impacto de las nuevas variables sobre el rendimiento de un modelo predictivo es un paso esencial en todo flujo de trabajo profesional. Esta evaluación responde a una pregunta clave: ¿Las nuevas características aportan una mejora significativa en la capacidad del modelo para generalizar?

Desde un enfoque científico, este proceso puede formalizarse como una prueba de hipótesis. La hipótesis nula (H_0) sostiene que las nuevas variables no mejoran el modelo. El objetivo es recolectar evidencia (a través de métricas de rendimiento) que permita rechazar esta hipótesis.

Esto puede hacerse de manera resumida siguiendo los pasos:

- ▶ Definición de un modelo de referencia (baseline). Se entrena un modelo (por ejemplo, un Random Forest o un XGBoost), utilizando únicamente el conjunto de características disponibles inicialmente. Este modelo se evalúa en un conjunto de prueba (hold-out o validación cruzada) mediante métricas como accuracy, F1, AUC o RMSE, según la naturaleza del problema.
- ▶ Construcción del modelo enriquecido. Se entrena el mismo modelo, con los mismos hiperparámetros y configuración, pero ahora incorporando las nuevas características generadas mediante ingeniería (manual o con apoyo de IA generativa). Es importante que todo lo demás permanezca constante para que la comparación sea válida.

- Medición del uplift. El impacto se mide como el aumento en la métrica de evaluación; por ejemplo:

$$Uplift = F1_{enriquecido} - F1_{baseline} .$$

Esta diferencia debe ser consistente en múltiples segmentos de validación o datasets temporales. Un uplift importante (por ejemplo, +0,07 en AUC o +0,10 en F1) puede justificar la incorporación de las nuevas variables.

En esta línea de trabajo, como ya se mencionó, la IA generativa contribuye en dos niveles. Primero, la sugerencia de evaluaciones, donde los LLM pueden recomendar las métricas más adecuadas según el tipo de problema (clasificación binaria, multiclase, regresión, etc.). Segundo, en la interpretación automática, es decir, al analizar los resultados de múltiples modelos, un LLM puede generar un informe en lenguaje natural que compare los rendimientos y sugiera decisiones (ej. «La inclusión de la variable riesgo_sector_sintética mejora la capacidad del modelo para detectar siniestros de alto costo en un +8 % de recall en el decil superior.»).

La evaluación de impacto debe considerar algunos aspectos que complementan y perfilan esta tarea; por ejemplo, que, si bien un pequeño uplift puede parecer insignificante en métricas agregadas, su valor de negocio puede ser muy alto cuando se enfoca en subgrupos clave (ej. clientes de alto valor o segmentos de riesgo). Por otro lado, la evaluación debe ser transparente, reproducible y complementada con análisis de importancia de características. Cabe mencionar que este enfoque también se utiliza para validar técnicas como target encoding, embeddings generativos o features condicionales diseñadas con IA generativa.

En resumen, más allá de medir la importancia de cada variable, reiteramos que es fundamental evaluar cómo cambian las métricas del modelo al añadir las variables generadas. Siguiendo con el ejemplo de los apartados anteriores, se entrenarán dos modelos:

- ▶ Baseline: con variables originales (antigüedad_meses, consumo_gb, llamadas_soporte, plan_contratado).
- ▶ Enriquecido: con las mismas variables más friccion_reciente y sentimiento.

```
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

# Modelo baseline
X_base =
pd.get_dummies(df[["antigüedad_meses", "consumo_gb", "llamadas_soporte", "plan_contratado"]],
drop_first=True)
y = df["churn"]

X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(X_base, y, test_size=0.3, random_state=42)
rf_base = RandomForestClassifier(random_state=42)
rf_base.fit(X_train_b, y_train_b)
y_pred_b = rf_base.predict(X_test_b)

# Modelo enriquecido
X_full = pd.get_dummies(df.drop(columns=["churn"]), drop_first=True)
X_train_f, X_test_f, y_train_f, y_test_f = train_test_split(X_full, y, test_size=0.3, random_state=42)
rf_full = RandomForestClassifier(random_state=42)
rf_full.fit(X_train_f, y_train_f)
y_pred_f = rf_full.predict(X_test_f)

# Métricas
results = pd.DataFrame({
    "Modelo": ["Baseline", "Enriquecido"],
    "Accuracy": [accuracy_score(y_test_b, y_pred_b), accuracy_score(y_test_f, y_pred_f)],
    "F1": [f1_score(y_test_b, y_pred_b), f1_score(y_test_f, y_pred_f)],
    "AUC": [roc_auc_score(y_test_b, rf_base.predict_proba(X_test_b)[:,1]),
            roc_auc_score(y_test_f, rf_full.predict_proba(X_test_f)[:,1])]
})
```

Results

Un escenario típico de salida es como el que se muestra en la tabla 5.

Modelo	Accuracy	F1	AUC
Baseline	0.78	0.62	0.74
Enriquecido	0.84	0.70	0.82

Tabla 5. Métricas ejemplo. Fuente: elaboración propia.

Así, los resultados confirman que la inclusión de variables generadas mejora sensiblemente el rendimiento del modelo. La ganancia en F1 y AUC muestra que el clasificador se vuelve más preciso al identificar casos de abandono.

La comparación subraya un aprendizaje central: no basta con generar variables, hay que evaluar su efecto en la predicción. El caso demuestra que las variables propuestas (fricción_reciente y sentimiento) no solo enriquecen el dataset, sino que aportan información útil para mejorar las métricas de negocio.

5.7. Feature engineering en pipelines de machine learning

En entornos reales, donde los modelos de Machine Learning se entrenan, validan y despliegan de forma continua, la ingeniería de características no puede limitarse a un cuaderno de Jupyter ni a scripts aislados. Debe formar parte de un pipeline estructurado, reproducible y controlado, que encapsule todas las transformaciones de datos desde la entrada cruda hasta la predicción.

Un pipeline de ML se implementa típicamente como un grafo acíclico dirigido (DAG) donde cada nodo representa una transformación (por ejemplo, imputación, codificación, generación de nuevas variables) y cada borde una dependencia entre operaciones. Dentro de este flujo, la ingeniería de características, tanto tradicional como asistida por IA generativa, ocupa un papel estratégico. Transforma los datos operacionales en representaciones que maximicen la capacidad del modelo para generalizar.

Integrar la ingeniería de características en un pipeline automatizado es crítico para tres aspectos:

- ▶ Evitar la fuga de datos (data leakage): todas las transformaciones deben aprenderse exclusivamente a partir de los datos de entrenamiento. Por ejemplo, si calculamos la media para imputar valores faltantes, debe ser la media del train set, no del conjunto completo.
- ▶ Asegurar la reproducibilidad: al encapsular todas las transformaciones en una secuencia programada, cualquier entrenamiento posterior aplicará exactamente las mismas reglas.

- ▶ Evitar el «training-serving skew»: lo que se transforma al entrenar debe transformarse idénticamente al hacer predicciones en producción. Si esto no se cumple, el modelo fallará silenciosamente. Recuerda que training-serving skew ocurre cuando los datos o las transformaciones aplicadas durante el entrenamiento del modelo no coinciden exactamente con los utilizados en producción (serving). El modelo parece funcionar bien durante el desarrollo, pero falla al hacer predicciones reales porque está viendo datos diferentes (en forma o contenido) a los que aprendió.

Veamos algunos ejemplos de cómo se puede materializar la ingeniería de características dentro de un pipeline:

Ejemplo: transformaciones tradicionales

En un pipeline de predicción de rotación de clientes (churn), una primera etapa incluye:

- ▶ Imputación de ingresos con un `IterativeImputer`.
- ▶ Generación de `gasto_por_visita` a partir de `total_gasto` dividido por `n_visitas`.
- ▶ Inclusión de una nueva variable sugerida por un LLM, `riesgo_textual_llamada`, generada a partir de la transcripción de llamadas, clasificada por un modelo basado en lenguaje natural.
- ▶ Estas operaciones se integran como nodos dentro del flujo usando librerías como `scikit-learn Pipeline`, `MLflow` o `TFX`.

Ejemplo: embeddings generativos en producción

Para una variable categórica como ocupación_cliente, el pipeline podría incluir:

- ▶ Representación inicial como índice entero.
- ▶ Paso por un Autoencoder entrenado previamente para generar un embedding de diez dimensiones.
- ▶ Uso de este embedding como entrada directa al modelo XGBoost.

La arquitectura del Autoencoder se entrena y se encapsula como un paso reutilizable dentro del pipeline final.

Ejemplo: generación dinámica de variables con LLM integrado en pipelines

En entornos profesionales donde la mejora continua del modelo es esencial, pueden integrarse agentes de IA basados en LLM (por ejemplo, a través de LangChain, CrewAI u otras plataformas) como componentes del pipeline de Machine Learning. Estos agentes actúan como coingenieros de características, capaces de analizar el dataset actual y sugerir nuevas variables derivadas que podrían mejorar el rendimiento predictivo. La lógica puede programarse para activarse automáticamente durante fases de validación iterativa o cuando el rendimiento del modelo no supere un umbral mínimo. El agente, al recibir la descripción del dataset y el desempeño del modelo, propone transformaciones útiles fundamentadas en razonamiento estadístico y conocimiento de dominio.

Pseudocódigo del agente generador dentro de un pipeline:

```
agent.input("Dataset: registros de visitas de clientes. Target: conversión a venta.")
agent.prompt("¿Qué nuevas variables derivadas podrían mejorar el modelo?")
agent.output("Sugerencia: Crear la variable ratio_visitas_último_mes_sobre_total = visitas_último_mes / visitas_totales")
```

Este tipo de variable, sugerida por el LLM, integra lógica de negocio: la intensidad reciente del interés del cliente como predictor. Al incluirla como nodo funcional en el pipeline, esta variable puede generarse automáticamente y evaluarse en tiempo real junto con otras variables.

El beneficio es doble; se aceleran las iteraciones de mejora del modelo y se promueve la exploración automatizada, sin depender exclusivamente del juicio humano en cada ciclo. Este tipo de integración marca el paso de pipelines estáticos a flujos de aprendizaje adaptativos y aumentados por IA generativa.

5.8. Comparación con técnicas tradicionales

A lo largo de este tema, hemos explorado cómo la IA Generativa transforma el proceso de ingeniería de características. Muchas tareas que antes requerían experiencia manual, intuición de dominio y tiempo extensivo ahora pueden abordarse con la ayuda de modelos de lenguaje capaces de razonar, transformar, sintetizar y explicar.

Aunque ya se han presentado ejemplos específicos, en esta sección mediante la tabla 6 sintetizamos las principales diferencias entre métodos clásicos y enfoques con IA generativa, destacando ventajas clave.

Tarea de <i>feature engineering</i>	Método tradicional	Método con IA generativa	Ventajas de la IA generativa
Crear interacciones.	Combinaciones manuales (ej. edad * ingresos).	Prompt: «Sugiere 3 interacciones de variables que podrían ser predictivas».	Descubre relaciones no obvias; incorpora conocimiento de dominio implícito.
Codificar variables categóricas.	One-Hot Encoding o Label Encoding.	Embeddings generativos aprendidos mediante Autoencoders o LLM	Captura semántica, reduce dimensionalidad y mejora la generalización.
Transformar texto libre.	Bag-of-Words, TF-IDF.	Extracción de entidades, análisis de sentimientos, clasificación contextual con LLMs	Traduce lenguaje natural en variables predictivas con alto poder explicativo.
Evaluar importancia de variables.	Árboles de decisión, permutaciones, SHAP, Gain.	Prompt: «¿Qué variables parecen más relevantes según la lógica del negocio y el modelo?» + interpretación con SHAP.	Asistencia en interpretación, priorización de variables y generación de reportes ejecutivos.

Tabla 6. Comparativa entre técnicas tradicionales y métodos de ingeniería de características con IA generativa. Fuente: elaboración propia.

5.9. Exploración práctica

Este caso busca reproducir un flujo de ingeniería de características aplicado a la predicción de churn en clientes de telecomunicaciones, integrando un LLM dentro de un pipeline de Machine Learning para crear nuevas variables, explicar su relevancia y medir su impacto. Su objetivo es demostrar cómo puede integrarse un modelo generativo (LLM) directamente en el pipeline de Machine Learning para proponer nuevas variables derivadas de forma automática, incorporar embeddings categóricos compactos, evaluar su impacto real en el rendimiento del modelo y generar explicaciones ejecutivas sobre la importancia de cada variable.

Se espera observar una mejora cuantificable, por ejemplo, en la métrica F1 gracias al enriquecimiento del dataset, junto con una narrativa generada automáticamente que contextualiza los principales factores que explican el churn, incluyendo la contribución de las nuevas variables creadas por el modelo generativo.

```
import os
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder, FunctionTransformer
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score, classification_report
from sklearn.inspection import permutation_importance
from sklearn.base import BaseEstimator, TransformerMixin

from openai import OpenAI

# -----
# 0) Configuración de API (LLM real, sin fallback)
# -----
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
```

```
# -----
# 1) Datos simulados
# -----
np.random.seed(42)
n = 300
df = pd.DataFrame({
    "id_cliente": [f"C-{i:05d}" for i in range(n)],
    "antiguedad_meses": np.random.randint(1, 72, size=n),
    "consumo_gb": np.random.gamma(shape=5.0, scale=6.0, size=n).round(2),
    "llamadas_soporte": np.random.poisson(lam=1.8, size=n),
    "plan_contratado": np.random.choice(["Basico", "Estandar", "Premium"], size=n, p=[0.5, 0.25, 0.25]),
})

# Objetivo con dependencia ligera de plan y fricción (llamadas)
logit = (
    -1.0
    + 0.015 * (df["consumo_gb"] < 15).astype(int)
    + 0.02 * (df["llamadas_soporte"] >= 3).astype(int)
    + 0.30 * (df["plan_contratado"].eq("Basico")).astype(int)
    - 0.20 * (df["plan_contratado"].eq("Premium")).astype(int)
)
p = 1 / (1 + np.exp(-logit))
df["churn"] = (np.random.rand(n) < p).astype(int)

TARGET = "churn"
CAT_COLS = ["plan_contratado"]
NUM_COLS = ["antiguedad_meses", "consumo_gb", "llamadas_soporte"]

# -----
# 2) Nodo LLM: ingeniería de variables (dentro del Pipeline)
# -----
def _strip_code_fences(text: str) -> str:
    """Limpia ```...``` si el LLM devuelve el código envuelto en fences."""
    t = text.strip()
    if t.startswith("```"):
        lines = t.splitlines()
        if lines and lines[0].startswith("```"):
            lines = lines[1:]
        if lines and lines[-1].startswith("```"):
            lines = lines[:-1]
        return "\n".join(lines).strip()
    return t
```

```
def generar_features_con_llm(X: pd.DataFrame) -> pd.DataFrame:
    """
    Paso del pipeline: pide al LLM 2 nuevas variables (sin I/O, sin objetivo),
    ejecuta el código pandas devuelto sobre X y retorna X enriquecido.
    """
    X = X.copy()
    prompt = f"""
Eres un científico de datos. Dataset de churn con columnas numéricas: {NUM_COLS} y categóricas:
{CAT_COLS}.
Devuelve EXCLUSIVAMENTE código pandas (sin markdown) que agregue EXACTAMENTE 2 nuevas
columnas al DataFrame X.
Evita data leakage (no uses la variable objetivo), y prohíbe I/O, imports, eval/exec o accesos a
sistema.
Sugerencias: ratios, interacciones simples, umbrales lógicos robustos.
"""
    rsp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.0
    )
    code = _strip_code_fences(rsp.choices[0].message.content)

    # Validación mínima de seguridad (en producción, endurecer con sandbox)
    blocked = ["import ", "open(", "os.", "sys.", "subprocess", "eval(", "exec(", "___", "pd.read", "np.load"]
    if any(b in code for b in blocked):
        raise RuntimeError(f"El LLM devolvió código no permitido:\n{code}")

    local_vars = {"X": X, "np": np, "pd": pd}
    try:
        exec(code, {}, local_vars)
        X = local_vars["X"]
    except Exception as e:
        raise RuntimeError(f"Fallo ejecutando el código del LLM:\n{code}\n\nError: {e}") from e

    return X
```

```
# -----
# 3) Embeddings categóricos (SVD sobre co-ocurrencia)
# -----
class CategoricalSVDEmbedding(BaseEstimator, TransformerMixin):
    """
    Aprende un "embedding" denso para una variable categórica a partir de sus co-ocurrencias
    con variables numéricas discretizadas. No usa deep learning ni librerías externas:
    - Discretiza num_cols en bins (cuantiles).
    - Construye una matriz de co-ocurrencia (cat x bins-por-variable).
    - Aplica SVD y toma las primeras n_components dimensiones como embedding.
    Evita fugas: el mapeo se aprende SOLO en fit(X_train).
    """

    def __init__(self, cat_col: str, num_cols: list[str], n_bins: int = 5, n_components: int = 2,
random_state: int = 42):
        self.cat_col = cat_col
        self.num_cols = num_cols
        self.n_bins = n_bins
        self.n_components = n_components
        self.random_state = random_state

    def fit(self, X: pd.DataFrame, y=None):
        X = X.copy()
        if self.cat_col not in X.columns:
            raise ValueError(f"{self.cat_col} no está en X")

        # Discretización robusta por cuantiles (para cada variable numérica)
        bin_cols = []
        self.bin_edges_ = {}
        for col in self.num_cols:
            if col not in X.columns:
                continue
            # Cuantiles (n_bins) -> bordes de bins
            qs = np.linspace(0, 1, self.n_bins + 1)
            edges = np.unique(np.quantile(X[col].values, qs))
            # Evitar bins vacíos
            if len(edges) < 3:
                edges = np.unique(np.quantile(X[col].values, [0, 0.5, 1.0]))
            self.bin_edges_[col] = edges
            binned = pd.cut(X[col], bins=edges, include_lowest=True, duplicates="drop")
            bin_name = f"BIN_{col}"
            X[bin_name] = binned.astype(str)
            bin_cols.append(bin_name)
```



```

# Matriz de co-ocurrencias: cat x (todas las columnas binned)
cooc_frames = []
for bc in bin_cols:
    ct = pd.crosstab(X[self.cat_col], X[bc], normalize="index") # distribución condicional por
categoría
    # Renombrar columnas para identificar variable de origen
    ct.columns = [f"{bc}::{c}" for c in ct.columns]
    cooc_frames.append(ct)

if not cooc_frames:
    raise ValueError("No se pudieron construir co-ocurrencias (verifica num_cols).")

M = pd.concat(cooc_frames, axis=1).fillna(0.0).sort_index()
self.categories_ = M.index.tolist()
A = M.values # (n_categorías x n_features_binned)

# SVD: A = U S V^T -> usamos U S como embedding (varianza explicada por componente)
# Para estabilidad reproducible:
rng = np.random.default_rng(self.random_state)
# SVD completa (numpy)
U, S, Vt = np.linalg.svd(A, full_matrices=False)
k = min(self.n_components, U.shape[1])
self.components_ = k
self.embeddings_ = (U[:, :k] * S[:k]) # (n_categorías x k)

# Mapeo categoría -> embedding vector
self.cat_to_vec_ = {
    cat: self.embeddings_[i, :].astype(float)
    for i, cat in enumerate(self.categories_)
}

return self

def transform(self, X: pd.DataFrame) -> pd.DataFrame:
    X = X.copy()
    # Agregar columnas de embedding (rellenar con 0 si categoría no vista en fit)
    for j in range(self.components_):
        X[f"{self.cat_col}_embed_{j}"] = 0.0
    for i, cat in X[self.cat_col].items():
        vec = self.cat_to_vec_.get(cat)
        if vec is not None:
            for j in range(self.components_):
                X.at[i, f"{self.cat_col}_embed_{j}"] = vec[j]
    return X

```

```
# -----
# 4) Definición de Pipelines (baseline vs enriquecido)
# -----
# Baseline: sin LLM ni embeddings
pre_base = ColumnTransformer([
    ("num", StandardScaler(), NUM_COLS),
    ("cat", OneHotEncoder(handle_unknown="ignore"), CAT_COLS),
])
baseline_pipe = Pipeline([
    ("pre", pre_base),
    ("clf", RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1))
])

# Enriquecido: 1) LLM features, 2) embeddings SVD de la categoría, 3) preprocesamiento completo
feature_node = FunctionTransformer(generar_features_con_llm, validate=False)
embed_node = CategoricalSVDEmbedding(cat_col="plan_contratado",
                                     num_cols=NUM_COLS, n_bins=5, n_components=2, random_state=42)

# Ojo: como tras LLM podrían aparecer nuevas columnas numéricas, usamos selectores dinámicos
pre_enr = ColumnTransformer([
    ("num", StandardScaler(), make_column_selector(dtype_include=np.number)),
    ("cat", OneHotEncoder(handle_unknown="ignore"), make_column_selector(dtype_include=object)),
])

enriched_pipe = Pipeline([
    ("llm_features", feature_node), # crea 2 nuevas variables con LLM
    ("cat_embedding", embed_node), # añade columnas plan_contratado_embed_0/1
    ("pre", pre_enr),
    ("clf", RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1))
])

# -----
# 5) Entrenamiento, evaluación y uplift
# -----
X = df[NUM_COLS + CAT_COLS]
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# Baseline
baseline_pipe.fit(X_train, y_train)
y_pred_b = baseline_pipe.predict(X_test)
f1_b = f1_score(y_test, y_pred_b)
```

```
# Enriquecido
enriched_pipe.fit(X_train, y_train)
y_pred_e = enriched_pipe.predict(X_test)
f1_e = f1_score(y_test, y_pred_e)

print("=== Resultados ===")
print(f"F1 Baseline : {f1_b:.3f}")
print(f"F1 Enriquecido : {f1_e:.3f}")
print(f"Uplift ( $\Delta F1$ ) : {f1_e - f1_b:+.3f}")
print("\n--- Clasificación (modelo enriquecido) ---")
print(classification_report(y_test, y_pred_e))

# -----
# 6) Importancia de variables (Permutation Importance sobre el pipeline enriquecido)
# -----
pi = permutation_importance(
    estimator=enriched_pipe,
    X=X_test,
    y=y_test,
    n_repeats=15,
    random_state=42,
    scoring="f1"
)

# Nombres de columnas tras preprocesamiento
pre_fitted = enriched_pipe.named_steps["pre"]
feat_names = pre_fitted.get_feature_names_out()

fi = pd.DataFrame({
    "feature": feat_names,
    "importance_mean": pi.importances_mean,
    "importance_std": pi.importances_std
}).sort_values("importance_mean", ascending=False)

print("\n=== Feature Importance (top 12) ===")
print(fi.head(12).to_string(index=False))

# -----
# 7) Interpretación con LLM (texto)
# -----
head_md = fi.head(8).to_markdown(index=False)
prompt_importancia = f"""
Ranking de importancia (Permutation Importance) top 8:
{head_md}

```

Redacta un resumen ejecutivo (5-7 líneas) explicando por qué estas variables podrían explicar el churn, incluyendo la intuición detrás de la(s) variable(s) nuevas creadas por el LLM y el rol del embedding categórico.

No uses listas; solo párrafo.

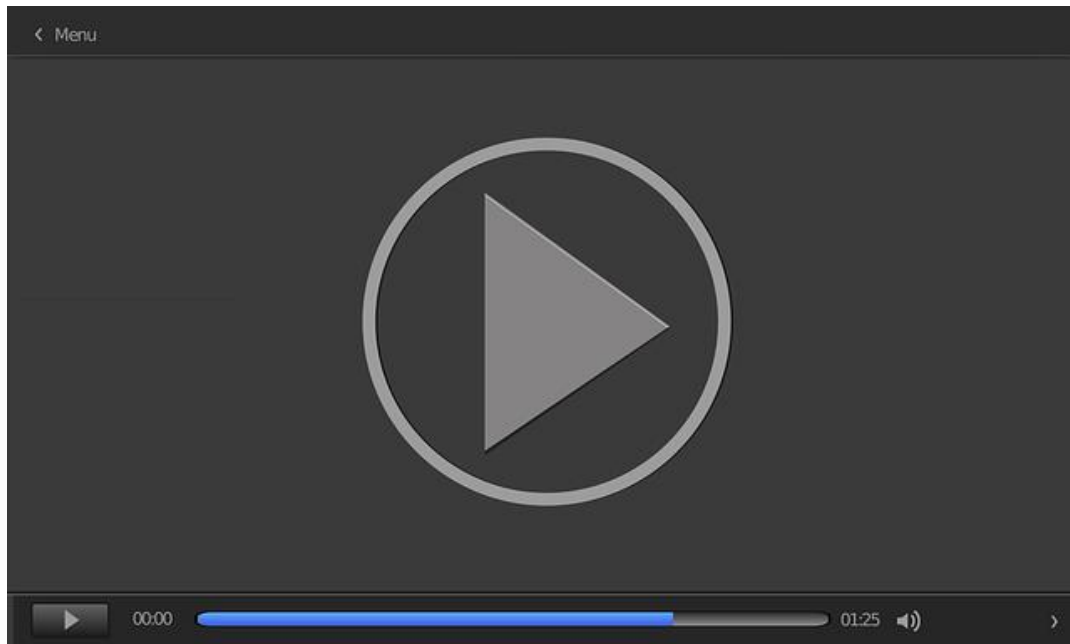
"""

```
rsp = client.chat.completions.create(  
    model="gpt-4o-mini",  
    messages=[{"role": "user", "content": prompt_importancia}],  
    temperature=0.2  
)  
print("\n--- Interpretación ejecutiva (LLM) ---")  
print(rsp.choices[0].message.content.strip())
```

Se reitera que la ingeniería de características asistida por IA generativa no reemplaza al experto, sino que expande su capacidad exploratoria, permitiéndole diseñar variables más robustas, adaptadas al contexto y validadas empíricamente en el flujo de modelado. Este enfoque abre nuevas oportunidades para acelerar la iteración, mejorar la calidad predictiva y estandarizar procesos complejos de preprocesamiento en entornos reales.

Durante la elaboración del tema, se empleó un modelo de IA generativa como herramienta de apoyo para optimizar la redacción y precisar conceptos técnicos. Su uso se llevó a cabo con responsabilidad y criterio académico mediante múltiples iteraciones de consulta, análisis y reformulación que solo con la debida experticia permitieron ajustar las respuestas con rigor y precisión al contexto específico del contenido desarrollado (OpenAI, 2025). Este uso complementario no sustituyó la consulta de fuentes académicas ni el ejercicio intelectual de construcción, sino que buscó reforzar la calidad y la claridad en el marco de una práctica académica transparente y ética.

En el vídeo, *Creación y evaluación de variables generadas por IA generativa*, veremos un ejemplo paso a paso de cómo generar nuevas características y medir su impacto sobre el rendimiento de un modelo predictivo. Incluye visualización de importancia de variables.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=12d13354-7183-482d-9d22-b382013e7c86>

5.10. Referencias bibliográficas

Boonstra, L. (2024). *Prompt Engineering*. Google.

Chollet, F. (2021). *Deep learning with Python*. Simon y Schuster.

Domingos, P. (2012). A few useful things to know about machine learning. *Communications of the ACM*, 55(10), 78–87.
<https://doi.org/10.1145/2347736.2347755>

Kuhn, M. y Johnson, K. (2019). *Feature Engineering and Selection: A Practical Approach for Predictive Models*. Chapman and Hall/CRC.
<https://doi.org/10.1201/9781315108230>

Lundberg, S. M. y Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 4765–4774.

Página de ChatGPT (<https://chat.openai.com/>).

Ingeniería de características automatizada con LLMs

Hollmann, N., Müller, S. y Hutter, F. (2023). Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering. *37th Conference on Neural Information Processing Systems*, 36, 44753-44775.

https://proceedings.neurips.cc/paper_files/paper/2023/file/8c2df4c35cdbee764ebb9e9d0acd5197-Paper-Conference.pdf

Este trabajo propone CAAFE, un enfoque que utiliza LLM para generar nuevas características a partir de la descripción del conjunto de datos y entregar tanto código como explicaciones semánticas. Es especialmente valioso porque combina el poder generativo del LLM con la explicabilidad en lenguaje natural y su código y demostraciones están disponibles públicamente.

Un enfoque de diálogo con LLMs para la creación de características

Ko, J., Park, G., Lee, D. y Lee, K. (2025). FeRG-LLM: Feature Engineering by Reason Generation Large Language Models. *ArXiv* 2503.23371. <https://arxiv.org/pdf/2503.23371?>

El artículo introduce FeRG-LLM, una arquitectura que emplea LLM con CoT (Chain of Thought), una serie de pasos lógicos para llegar a una solución a través de un diálogo en dos etapas para generar nuevas características y justificarlas. Se sugiere esta investigación por su profunda integración del razonamiento humano en el diseño automático de características, su foco en la eficiencia y su aplicabilidad en contextos con restricciones corporativas.